Practical Assessment Review Response

This document answers the required review questions for the Halo IT Data & AI Specialist practical assessment.

Requirement Fit Summary

Area	Response
Part 1: Source, Combine & Prepare	Completed with a reproducible Python pipeline, external NZ public-holiday enrichment, cleaned categories, reporting sub-categories, quality flags, CSV and Parquet outputs.
Part 2: Visualise	Completed as `power_bi/Dashboard.pbix`, using the prepared 2024-2025 reporting dataset for operational support analysis.
Part 3: AI Agent	Completed as a Docker-ready Streamlit + DuckDB analytics agent with guarded SQL, audit logging, business analysis, result tables, and charts.
Written review	Completed in this document.

1. Where Did The Datasets Come From, And Why Did You Choose Them?

The primary dataset is the provided synthetic Excel workbook, `TechSolve - Ticket Data.xlsx`. I used it as the source of truth for support tickets, including ticket creation and resolution dates, categories, statuses, priorities, SLA fields, customer context, teams, CSAT, and escalation data.

The external dataset is New Zealand national public holidays for 2023-2025. I chose public holidays because they can materially affect support operations: staffing levels, customer availability, backlog build-up, SLA performance, and ticket volumes around holiday windows. The project includes a seeded CSV at `data/raw/nz_public_holidays.csv` so the assessment remains runnable without an API key, plus `src/fetch_public_holidays.py` for refreshing from public-holidays.nz when an API key is available.

2. Describe Your Process For Working Through This Scenario

I started by profiling the raw workbook to understand row counts, available fields, category quality, dates, missing values, and SLA fields. The biggest early finding was that the source category labels were inconsistent, so I kept the raw/source category fields and added a separate reporting taxonomy derived from the issue descriptions.

For Part 1, I built a repeatable preparation script that parses dates, standardises source category aliases, derives reporting categories and sub-categories, recomputes SLA breach logic, flags data quality issues, removes direct personal identifiers from the reporting output, enriches tickets with public holiday context, and writes dashboard/agent-ready outputs.

For Part 2, I used the processed dataset as the basis for the Power BI dashboard. The dashboard is intended for an operations manager, so the focus is ticket issues, status, time to resolution, SLA performance, and holiday-related operational patterns rather than raw record browsing.

For Part 3, I built a read-only analytics agent over DuckDB. The agent turns natural-language questions into bounded analytical plans and SQL, validates SQL locally, executes against the prepared data, and returns a grounded answer with evidence, possible operational drivers, recommended actions, caveats, and provenance. I added audit logging so each query can be reviewed later, including the generated SQL and validated SQL.

I then added tests, evaluation cases, Docker support, and documentation so the work can be reproduced and reviewed without depending on one local machine.

3. What Tools And Programs Did You Use, And Why Those Over Alternatives?

I used Python and pandas for data preparation because they make the cleaning steps repeatable, testable, and easy to inspect. I wrote both CSV and Parquet outputs: CSV is easy for reviewers and Power BI, while Parquet preserves types better and is faster for analytical querying.

I used Power BI for the dashboard because it is a natural fit for the visualisation requirement and for an operations manager audience. It provides interactive filtering, KPI views, and quick exploration of 2024-2025 ticket performance.

I used DuckDB for the AI agent's analytical layer because the dataset is tabular and the expected questions are aggregation-heavy. SQL over DuckDB is more transparent and auditable for this scenario than a pure vector-search RAG approach.

I used Streamlit for the agent interface because it is lightweight, easy to run locally or in Docker, and suitable for a practical assessment demo. I used Pydantic to validate structured agent outputs, `sqlglot` to inspect and guard SQL, pytest for regression tests, ruff and mypy for code quality, and Docker to make the runtime reproducible.

The agent supports Groq-backed live mode, but deterministic fake mode is the default. This keeps tests and demos reliable and avoids accidental live LLM calls when a local API key is present.

4. What Would You Do Differently? What Did You Find Most Challenging?

With more time, I would add regional NZ anniversary days or weather data to the external enrichment, because national holidays alone do not capture every local staffing or customer-availability effect. I would also add a richer Power BI model documentation export, more dashboard user testing with an operations manager persona, and broader live LLM evaluation after confirming data egress approval.

The most challenging part was deciding how much to clean versus how much to flag. The raw category and SLA fields contain many inconsistencies, but simply overwriting or deleting questionable records would hide useful operational and data quality signals. I therefore preserved source fields, added calculated reporting fields, and surfaced mismatch flags.

The second challenge was making the AI agent useful without making it unsafe. The agent needs to answer flexible business questions, but it should not expose personal data, secrets, raw unrestricted notes, arbitrary files, or destructive SQL. I addressed this with input refusal rules, schema allowlists, SQL AST validation, read-only DuckDB connections, capped result sizes, and audit logs.

5. Roughly How Long Did You Spend On Each Phase?

These are rounded estimates:

Phase	Approximate time
Requirement review and data profiling	2 hours
Part 1 data cleaning, holiday enrichment, outputs, and tests	6 hours
Part 2 dashboard design and Power BI iteration	6 hours
Part 3 AI agent architecture, implementation, Docker, and UI	8 hours
Evaluation, security checks, documentation, and final review	4 hours
Total	26 hours

6. Did You Use AI Tools During This Practical? If So, How?

Yes. I used AI tools as a development assistant for requirement analysis, implementation support, test planning, documentation drafting, and code review. The AI assistance was used to accelerate the work, but the project decisions were still grounded in the dataset and validated with local checks.

Specifically, AI assistance helped with:

- interpreting the assessment requirements and breaking the work into Part 1, Part 2, Part 3, and written review deliverables;
- designing the reporting taxonomy and data quality flags;
- building and reviewing the Python preparation pipeline and tests;
- designing the AI agent architecture, safety controls, audit logging, and evaluation suite;
- improving documentation and explaining implementation trade-offs.

I did not treat AI output as automatically correct. The final project was validated with repeatable scripts, unit and integration tests, offline evaluation cases, Docker health checks, and manual review points.

Current Known Limitations

- The Power BI file is committed, but the internal PBIX visual layout should be opened once in Power BI Desktop for final visual QA before submission.
- A limited owner-approved live Groq smoke test was completed successfully. Broader live evaluation remains out of scope for this assessment. The repository defaults to fake mode and includes mocked Groq integration tests.
- Public holiday enrichment currently uses national holidays only; regional anniversary days are a future enhancement.
- The source data is intentionally noisy. The pipeline highlights category and SLA inconsistencies rather than hiding them.
- The v1 evaluation runner checks that refusal cases are refused and normal cases complete, but it does not yet fully assert every configured `expected_intent`, `expected_dimensions`, `required_metrics`, and `required_filters`. That would be the next enterprise-grade evaluation enhancement.

Validation Evidence

Most recent local validation:

```
python src/prepare_data.py             passed
python scripts/build_duckdb.py         passed
ruff check .                           passed
ruff format --check .                  passed
mypy agent                             passed
pytest --cov=agent --cov-report=term-missing
                                       42 passed, 77% coverage
python -m agent.evals.run --suite smoke
                                       51 passed
python -m agent.evals.run --suite security
                                       12 passed
bandit -r agent scripts                no issues identified
pip-audit                              no known vulnerabilities found
docker build --tag halo-it-agent:ci . passed
Groq live smoke test                   COMPLETED, quality score 0.981
```